Initial Product Backlog

The backlog is organised as **Epic → Feature → User Story → Task**.

- **Estimates:** Fibonacci story points (1, 2, 3, 5, 8); one point is approximately half a working day for one person.
- **Priority: M**ust, **S**hould, **C**ould.
- **Owners** are team members: Ali Kaan Demiralp (Lead Architect and DevOps), Murat Yatağan (Product Owner), Furkan Berkay Öztürk (Scrum Master), Duhan Kaan Şenol (Requirements and UX), Erol Yağız Gemalmaz (QA). Every member also contributes code.
- **Dependencies** are listed in each story (`deps`).

Total: Must = 73 points, Should = 16, Could = 8. Security stories US-01 to US-05 are not candidates for descoping; if velocity is lower than planned, Should and Could items are removed first.

Epics

Epic	Goal
E1 Accounts and security	Register, log in, roles, secure basics
E2 Scenario library	Content learners practise on
E3 Training session	Core learning loop and scoring
E4 Progress	Users see their history
E5 Admin dashboard	Admins manage content and see statistics
E6 Platform and quality	Docker, CI, tests, docs

Features

Feature	Epic	Stories
F1.1 Account registration and login	E1	US-01, US-02
F1.2 Access control and abuse protection	E1	US-03, US-04, US-05
F2.1 Scenario content	E2	US-06, US-07
F3.1 Training session lifecycle	E3	US-08, US-10, US-11
F3.2 Learner interface	E3	US-09, US-12
F4.1 Score history	E4	US-13
F5.1 Scenario management	E5	US-14, US-15, US-17
F5.2 Administrator statistics	E5	US-16
F6.1 Deployment and delivery pipeline	E6	US-18, US-19, US-22
F6.2 Quality assurance	E6	US-20, US-23
F6.3 API tooling	E6	US-21
F7.1 Optional enhancements	E1, E3	US-24, US-25

User stories

E1 Accounts and security

US-01 Register an account · M · 3 pts · Ali Kaan Demiralp · deps: none As a visitor I want to create an account so that my progress is saved.

- Given valid username, email and a 12+ char password, when I register, then I get 201 and the account has role `learner`.
- Given a duplicate username or email, then I get 409 and no account is created.
- Given an invalid email or short password, then I get 400 with a clear reason.
- Password is stored hashed (Argon2id/bcrypt), never returned or logged.

US-02 Log in and out · M · 3 pts · Ali Kaan Demiralp · deps: US-01 As a user I want to log in so that I can access my sessions.

- Valid credentials return an access token with expiry.
- Wrong credentials return 401 with the *same* message for unknown user and wrong password.
- Protected endpoints return 401 without a valid token.

US-03 Role-based access control · M · 3 pts · Ali Kaan Demiralp · deps: US-02 As a system owner I want admin features restricted to admins so that learners cannot change content or see other users' data.

- A learner calling any `/admin/**` endpoint gets 403.
- A learner cannot read another user's results (tested for IDOR).

US-04 Rate limiting on auth endpoints · M · 2 pts · Ali Kaan Demiralp · deps: US-02 As a system owner I want repeated failed logins to be rate-limited so that password guessing is slowed down.

- More than 5 failed logins per minute per IP/account returns 429 with `Retry-After`.
- Limit is configurable by environment variable and covered by a test.

US-05 Request validation and safe errors · M · 3 pts · Ali Kaan Demiralp · deps: OpenAPI draft (`api/openapi.yaml`) As a user I want invalid requests rejected with safe, clear errors so that the system never leaks internal details or runs injected scripts.

- Requests violating the OpenAPI schema (wrong type, extra fields, over-length) return 400.
- No stack traces or SQL errors reach clients; errors use problem+json.
- Scenario text is rendered escaped in the UI (XSS test: `<script>` shows as text).

E2 Scenario library

US-06 Seed scenario library · M · 5 pts · Murat Yatağan · deps: US-14 As a trainer I want at least 30 ready-made scenarios so that the demo and the first training sessions are credible.

- 30 scenarios: about 60 % phishing and 40 % legitimate, mix of easy/medium/hard.
- Each has sender, subject, body, indicators, explanation. Only fictional brands and reserved domains (`.test`, `example.com`).
- Loadable by a seed script run by docker-compose.

US-07 Indicator vocabulary · S · 2 pts · Murat Yatağan · deps: none As a learner I want red flags to be named consistently with a plain description so that the feedback is easy to understand.

- A fixed list of red-flag codes with a human-readable description each, used by both UI and API.

E3 Training session

US-08 Start a session · M · 5 pts · Ali Kaan Demiralp · deps: US-02, US-06 As a learner I want to start a session of N scenarios so that I can practise in short rounds.

- Start returns 10 random scenarios by default (1 to 20 allowed).

- Scenarios in the response contain **no** answer key or explanation.
- If fewer scenarios exist than requested, return 422.

US-09 View a scenario in the UI · M · 5 pts · Duhan Kaan Şenol · deps: US-08 As a learner I want each scenario to look like a real inbox message so that I practise in realistic conditions.

- The email is shown like an inbox message (sender name, address, subject, body, link labels with the real target shown as text).
- Links are never clickable.
- Works at laptop and phone widths.

US-10 Submit a verdict and get feedback · M · 5 pts · Ali Kaan Demiralp · deps: US-08 As a learner I want to submit my verdict and get instant feedback so that I learn which red flags I missed.

- Learner chooses *Phishing* or *Legitimate*, optionally ticks suspicious indicators.
- Response says correct or incorrect, shows the correct verdict, missed indicators and the explanation.
- The same scenario cannot be answered twice in a session (409).
- A learner cannot answer in someone else's session (403).

US-11 Scoring and session result · M · 3 pts · Ali Kaan Demiralp · deps: US-10 As a learner I want my session to be scored and saved so that I know how well I did.

- Score = correct verdicts / total × 100, rounded; completing the session returns the result and stores it.
- Completing an already-completed session returns 409.

US-12 Session summary screen · M · 3 pts · Duhan Kaan Şenol · deps: US-11 As a learner I want a summary at the end of a session so that I can review what I missed.

- After the last scenario, the learner sees score, number correct and which scenarios were missed.

E4 Progress

US-13 Score history · M · 3 pts · Duhan Kaan Şenol · deps: US-11 As a learner I want to see my past scores and a trend so that I can track my progress.

- Learner sees a list (newest first) of own completed sessions with date and score.
- A simple trend indicator (improving / flat / declining vs. previous) is shown. A chart is a Could.

E5 Admin dashboard

US-14 Scenario data model and CRUD API · M · 5 pts · Ali Kaan Demiralp · deps: OpenAPI draft (`api/openapi.yaml`), US-03 As an admin I want a scenario API so that content can be managed without code changes.

- Admin can create, read, update, delete scenarios via the API; validation as per the OpenAPI contract.
- Admin changes record who changed what and when (audit fields).

US-15 Admin scenario management UI · M · 8 pts · Furkan Berkay Öztürk · deps: US-14, US-03 As an admin I want a scenario management screen so that I can add, edit and delete scenarios without touching code.

- Admin sees a table of scenarios, can add/edit/delete via forms with validation messages.
- Deleting asks for confirmation.
- A learner never sees this page.

US-16 Admin statistics · S · 5 pts · Murat Yatağan · deps: US-11, US-03 As an admin I want aggregate statistics so that I can see how learners perform and which scenarios are hardest.

- Admin sees total users, total sessions, average score, and the 5 hardest scenarios (lowest correct rate).
- No personal answer data per user is shown.

US-17 Bootstrap first admin · M · 2 pts · Ali Kaan Demiralp · deps: US-01 As an operator I want the first admin to be created from environment variables so that no default password ships in the code.

- The first admin is created from environment variables at startup (no default password in code).
- Documented in README.

E6 Platform and quality

US-18 One-command Docker deployment · M · 5 pts · Ali Kaan Demiralp · deps: none (sets up the `api/`, `web/` and `deploy/` skeleton) As a grader or developer I want one command to start the whole system so that it runs the same way on any machine.

- `docker compose up` starts web, api and db and seeds data; the app works at `http://localhost:8080` from a clean checkout.
- Secrets come from `.env`, not the repository.

US-19 CI pipeline · M · 5 pts · Ali Kaan Demiralp · deps: US-18 As a team member I want CI checks on every pull request so that broken or untested code cannot be merged.

- GitHub Actions runs lint, unit tests and build on every PR; required by branch protection.
- Coverage reported; pipeline fails below 50 %.

US-20 Test suite and test plan · M · 5 pts · Erol Yağız Gemalmaz · deps: US-19; tests are added as US-01 to US-04, US-08, US-10 and US-11 are delivered As the QA engineer I want a test plan and automated tests so that core flows and security rules are verified.

- Test plan document, unit tests for scoring and validation, integration tests for auth and session flow, at least 50 % coverage.
- Includes security tests: wrong role → 403, rate limit → 429, answer key not leaked in session start.

US-21 OpenAPI mock and contract check · S · 3 pts · Ali Kaan Demiralp · deps: OpenAPI draft (`api/openapi.yaml`), US-19 As a frontend developer I want a mock server generated from the OpenAPI contract so that UI work can start before the API is ready.

- Mock server from `openapi.yaml` lets the UI work before the API is ready; lint runs in CI.

US-22 Dependency and licence scanning · S · 3 pts · Ali Kaan Demiralp · deps: US-19 As the team we want dependency and licence scanning so that vulnerable or incompatible packages are caught early.

- Dependabot and an audit command run in CI; licence list generated for the NOTICE file.

US-23 Static analysis (SonarQube) report · S · 3 pts · Erol Yağız Gemalmaz · deps: US-19 As the team we want a SonarQube report so that technical debt is measured for the final submission.

- SonarQube scan produces the technical-debt report for the final submission.

Could-have (only after the MVP works)

US-24 Forgot-password flow · C · 5 pts · Murat Yatağan · deps: US-02, optional email stub As a learner I want to reset a forgotten password so that I can regain access to my account.

- The reset flow works through the optional local email stub; no real email is sent.

US-25 Difficulty filter at session start · C · 3 pts · Erol Yağız Gemalmaz · deps: US-08, US-06 As a learner I want to choose a difficulty level so that sessions match my skill.

- Session start accepts an optional difficulty filter (easy / medium / hard).

Tasks

Tasks are the work items created as GitHub issues and linked to their parent story. Estimates are set at Sprint Planning.

Task	Story	Description	Owner
T-01	US-01	Design users table and migration	Ali Kaan Demiralp
T-02	US-01	Implement register endpoint with Argon2id password hashing	Ali Kaan Demiralp
T-03	US-01	Build registration form	Duhan Kaan Şenol
T-04	US-01	Write registration unit and integration tests	Erol Yağız Gemalmaz
T-05	US-02	Implement login endpoint and token issuing	Ali Kaan Demiralp
T-06	US-02	Build login form and token handling	Duhan Kaan Şenol
T-07	US-02	Write authentication tests	Erol Yağız Gemalmaz
T-08	US-03	Implement role-check middleware	Ali Kaan Demiralp
T-09	US-03	Write role and IDOR tests	Erol Yağız Gemalmaz
T-10	US-04	Implement rate-limiting middleware for authentication endpoints	Ali Kaan Demiralp
T-11	US-04	Write rate-limit tests	Erol Yağız Gemalmaz
T-12	US-05	Implement request validation from the OpenAPI schemas	Ali Kaan Demiralp
T-13	US-05	Implement problem+json error handler	Ali Kaan Demiralp
T-14	US-05	Write XSS escaping test	Erol Yağız Gemalmaz
T-15	US-06	Write 30 scenarios with indicators and explanations	Murat Yatağan
T-16	US-06	Implement seed script	Murat Yatağan
T-17	US-06	Review scenarios for fictional domains and realism	Furkan Berkay Öztürk
T-18	US-07	Define indicator vocabulary and constants	Murat Yatağan
T-19	US-08	Implement scenario selection logic	Ali Kaan Demiralp
T-20	US-08	Implement start-session endpoint without answer key	Ali Kaan Demiralp
T-21	US-08	Write answer-key leakage test	Erol Yağız Gemalmaz
T-22	US-09	Build scenario view component	Duhan Kaan Şenol
T-23	US-09	Make the layout responsive	Duhan Kaan Şenol
T-24	US-10	Implement submit-answer endpoint	Ali Kaan Demiralp
T-25	US-10	Build verdict controls and feedback panel	Duhan Kaan Şenol

Task	Story	Description	Owner
T-26	US-10	Write duplicate-answer and ownership tests	Erol Yağız Gemalmaz
T-27	US-11	Implement scoring and complete-session endpoint	Ali Kaan Demiralp
T-28	US-11	Write scoring unit tests	Erol Yağız Gemalmaz
T-29	US-12	Build session summary screen	Duhan Kaan Şenol
T-30	US-13	Implement results history endpoint	Ali Kaan Demiralp
T-31	US-13	Build score history view	Duhan Kaan Şenol
T-32	US-14	Implement scenario schema and CRUD endpoints	Ali Kaan Demiralp
T-33	US-14	Add audit fields	Ali Kaan Demiralp
T-34	US-14	Write CRUD tests	Erol Yağız Gemalmaz
T-35	US-15	Build scenario table view	Furkan Berkay Öztürk
T-36	US-15	Build create and edit form with validation	Furkan Berkay Öztürk
T-37	US-15	Implement delete confirmation	Furkan Berkay Öztürk
T-38	US-16	Implement statistics query and endpoint	Murat Yatağan
T-39	US-16	Build statistics view	Murat Yatağan
T-40	US-17	Implement first-administrator bootstrap from environment variables	Ali Kaan Demiralp
T-41	US-18	Write Dockerfiles	Ali Kaan Demiralp
T-42	US-18	Write docker-compose configuration and seeding	Ali Kaan Demiralp
T-43	US-18	Verify start-up on a clean machine	Erol Yağız Gemalmaz
T-44	US-19	Write GitHub Actions workflow for lint, test and build	Ali Kaan Demiralp
T-45	US-19	Add coverage gate at 50 %	Ali Kaan Demiralp
T-46	US-20	Write test plan	Erol Yağız Gemalmaz
T-47	US-20	Write end-to-end test of the main user journey	Erol Yağız Gemalmaz
T-48	US-21	Configure OpenAPI mock server and lint step	Ali Kaan Demiralp
T-49	US-22	Configure Dependabot and dependency audit step	Ali Kaan Demiralp
T-50	US-23	Set up SonarQube scan and produce the report	Erol Yağız Gemalmaz
T-51	Process	Maintain backlog priorities and refine stories each sprint	Murat Yatağan
T-52	Process	Maintain GitHub Projects board, stand-up notes and retrospectives	Furkan Berkay Öztürk
T-53	Process	Write Software Requirements Specification and non-functional requirements (Sprint 2)	Duhan Kaan Şenol
T-54	Process	Prepare Figma prototypes and user-flow diagrams (Sprint 2)	Duhan Kaan Şenol

Task	Story	Description	Owner
T-55	Process	Write C4 and UML architecture documents (Sprint 2)	Ali Kaan Demiralp

Stories US-24 and US-25 are broken into tasks when they are pulled into a sprint.

Summary

Epic	Stories	Points
E1 Accounts and security	US-01 to 05	14
E2 Scenario library	US-06, 07	7
E3 Training session	US-08 to 12	21
E4 Progress	US-13	3
E5 Admin	US-14 to 17	20
E6 Platform and quality	US-18 to 23	24
Could	US-24, 25	8
Total	25 stories	**97**

Suggested sprint slicing

- **Sprint 2 (docs-heavy, W3-5):** SRS, UI prototypes, C4/UML; code skeleton, US-18 (Docker skeleton), US-01/02/14 started.
- **Sprint 3 (W6-8, MVP):** US-03 to 13, US-15, US-17, US-19, US-20 (core).
- **Final (W9-10):** US-16, 21-23, hardening, demo rehearsal.

Traceability

Each story maps to an OpenAPI `operationId` where applicable: US-01 `registerUser`, US-02 `login`, US-08 `startSession`, US-10 `submitAnswer`, US-11 `completeSession`, US-13 `listMyResults`, US-14 `adminCreateScenario` / `adminUpdateScenario` / `adminDeleteScenario` / `adminListScenarios`, US-16 `adminGetStats`. Keep the issue number in PR titles so the trace persists to code.